Developer Guide

Credit Default Prediction: Explainable AI & Bias Mitigation QM640 Capstone Project

Contents

1. Overview

This project implements a credit-default scoring pipeline that is simultaneously **accurate**, **explainable**, and **fairness-audited**. It targets two datasets with two different roles:

Dataset	Role	Target
Home Credit Default Risk (Kaggle)	Full pipeline: EDA, training, tuning, evaluation, explainability, fairness audit, bias mitigation	`TARGET` (loan default, binary)
HMDA (CFPB mortgage data)	Fairness audit **only**, on an approval/denial model	`action_taken_binary` (loan originated vs. denied)

HMDA is deliberately *not* used to build a second "default" model. HMDA records the mortgage **origination decision**, not loan performance, so treating it as a default-prediction task would conflate two different outcomes. Its sole purpose here is to test whether the fairness-auditing methodology built for Home Credit generalizes to a second, independently-labeled dataset and decision task.

This scope (one full pipeline + one fairness-audit-only pipeline, one mitigation technique) is a deliberate MVP reduction from a larger original proposal (two full pipelines × two model families × three mitigation techniques), made because the larger matrix was judged infeasible for a single capstone term.

Data availability

No Kaggle account/API credentials were available at build time. The pipeline therefore runs by default against a **schema-accurate synthetic data generator** (`src/dac/data/synthetic.py`) that mirrors the real datasets' column names, dtypes, value domains, and missingness patterns. A small, explicitly documented synthetic bias term is injected on the protected attributes so the fairness-audit and mitigation stages have a real, known effect to detect and correct. Real-data download scripts are included and the loader prefers real data automatically the moment it's present — see Section 4.3.

---

2. High-Level Design

2.1 Pipeline architecture

The system is a linear, ten-stage pipeline orchestrated by `scripts/run_pipeline.py`. Every stage is also independently importable from the `dac` package, and each is mirrored by one notebook under `notebooks/` for interactive/exploratory use.

```
1. Load data dac.data.loader (real file if present, else synthetic)
2. EDA dac.eda.eda_report -> reports/figures/, eda_*.md
3. Feature engineering dac.features.engineering -> engineered DataFrame + train/test split
4. Baseline training dac.models.train -> Logistic Regression, XGBoost
5. Hyperparameter tuning dac.models.tune -> RandomizedSearchCV (LR), Optuna/TPE (XGBoost)
6. Evaluation dac.models.evaluate -> ROC-AUC, PR-AUC, F1, KS, Brier + plots
7. Explainability dac.explainability.shap_explain -> SHAP global + local plots
8. Fairness audit dac.fairness.audit -> Fairlearn group metrics (pre-mitigation)
9. Bias mitigation dac.fairness.mitigation -> reweighing, retrain, re-audit (post-mitigation)
10. HMDA fairness audit dac.fairness.hmda_audit -> steps 1-2 + 4 + 8, restricted to HMDA
```

Every stage reads from and writes to a single `CONFIG` dict (`dac.config.CONFIG`, loaded from `config/config.yaml`) so paths, seeds, model hyperparameter ranges, and fairness thresholds are defined in exactly one place.

2.2 Module responsibilities

Module	Responsibility
`dac.config`	Loads `config/config.yaml`, resolves all paths to absolute `Path` objects, creates directories on import.
`dac.data.synthetic`	Generates schema-accurate synthetic Home Credit / HMDA data, including a documented synthetic bias term.

`dac.data.loader`	Chooses real data (if downloaded) vs. synthetic fallback; caches synthetic data as parquet.
`dac.features.engineering`	Derives interpretable features (age, employment years, credit ratios); builds the shared `ColumnTransformer` preprocessing pipeline used by every model.
`dac.eda.eda_report`	Produces missingness, target-balance, correlation, and protected-attribute breakdown figures + a markdown report.
`dac.models.train`	Builds Logistic Regression / XGBoost `sklearn` `Pipeline`s (preprocessor + estimator) and fits them.
`dac.models.tune`	`RandomizedSearchCV` for Logistic Regression; Optuna (TPE sampler) for XGBoost, both optimizing CV ROC-AUC.
`dac.models.evaluate`	Threshold-free + threshold-based metrics (ROC-AUC, PR-AUC, F1, KS statistic, Brier score) and diagnostic plots (ROC, PR, calibration, confusion matrix).
`dac.explainability.shap_explain`	Global (bar, beeswarm) and local (waterfall) SHAP explanations for a fitted pipeline.
`dac.fairness.audit`	Fairlearn `MetricFrame`-based per-group audit: selection rate, demographic parity ratio/difference, equalized-odds difference, four-fifths-rule flag.
`dac.fairness.mitigation`	Kamiran & Calders (2012) reweighing: per-row sample weights that decorrelate a protected attribute from the label in the training set.
`dac.fairness.hmda_audit`	Wires together loading, feature engineering, training, and auditing for the HMDA-only fairness pipeline; excludes protected attributes from model features by design.

2.3 Key design decisions

- **Config-driven, not hard-coded.** All paths, the random seed, split ratios, model hyperparameter defaults, tuning trial counts, and the fairness disparate-impact threshold live in `config/config.yaml`. Nothing in `src/dac` hard-codes a file path.
- **One shared preprocessing pipeline per dataset.** `build_preprocessor()` returns a single `ColumnTransformer` (median-impute + scale for numeric, most-frequent-impute + one-hot for categorical) reused identically by Logistic Regression and XGBoost, so SHAP and fairness results are comparable across models.
- **Protected attributes are excluded from model features.** `CODE_GENDER` / `AGE_GROUP` (Home Credit) and `derived_race` / `derived_sex` (HMDA) are never passed to the estimator — they are used only for the post-hoc

fairness audit, matching standard fair-lending practice.

- **PR-AUC and Brier score alongside ROC-AUC.** Home Credit's ~8-10% default rate makes ROC-AUC alone potentially misleading under class imbalance.
- **Reweighing over in-processing/post-processing mitigation.** Chosen because it is model-agnostic (identical code path for Logistic Regression and XGBoost) and requires no architecture changes — the pragmatic MVP choice among reweighing, adversarial debiasing, and equalized-odds postprocessing.
- **The four-fifths rule is a borrowed convention, not a legal threshold.** The 0.80 disparate-impact-ratio flag in `dac.fairness.audit` is an EEOC employment-discrimination heuristic (US), with no direct legal standing under ECOA/Reg B or outside the US. It's used here as a common fairness-ML convention for flagging practically significant gaps.
- **Real-data-if-present, synthetic-otherwise loading.** `dac.data.loader` never requires code changes to switch from synthetic to real data — it checks `data/raw/home_credit/application_train.csv` and `data/raw/hmda/hmda_*.csv` first, falling back to a cached synthetic dataset only if those are absent.

2.4 Data flow

```
config/config.yaml --> dac.config.CONFIG --> every stage

Kaggle / CFPB API --> scripts/download_*.py --> data/raw/
|
v
dac.data.loader.load_*()
(real file, else synthetic)
|
v
dac.features.engineering (+ train/test split)
|
+----------------------+----------------------+
v v v
dac.models.train dac.models.tune dac.eda.eda_report
| | |
+----------+-----------+ v
v reports/figures/*.png
dac.models.evaluate reports/model_performance/*.md
|
+--------------------+--------------------+
v v v
dac.explainability dac.fairness.audit dac.fairness.mitigation
.shap_explain (pre-mitigation) --> retrain --> dac.fairness.audit
| | (post-mitigation)
v v v
reports/figures/ reports/model_performance/fairness/*.json
.../shap/ models/*.joblib
```

---

3. Project Structure

```
Code/
+-- config/
| +-- config.yaml # single source of truth: paths, seed, model/tuning/fairness params
+-- data/
| +-- raw/ # downloaded or cached-synthetic input data (gitignored)
| +-- processed/ # optional intermediate feature caches (gitignored)
| +-- external/ # supplementary reference data (gitignored)
| +-- README.md # expected layout + how to get real data
+-- docs/
| +-- DEVELOPER_GUIDE.md/.pdf # this document
+-- models/ # trained model artifacts (*.joblib), gitignored
+-- notebooks/ # 01-05, one per pipeline stage group
+-- reports/
| +-- figures/ # all generated PNGs, gitignored
| +-- model_performance/ # metrics CSV/JSON/MD, gitignored
+-- scripts/
| +-- download_home_credit.py # real Kaggle download (needs kaggle.json)
| +-- download_hmda.py # real CFPB HMDA download (no auth)
| +-- run_pipeline.py # end-to-end orchestrator -- the main entry point
+-- src/dac/ # the installable `dac` package
| +-- config.py
| +-- data/ (loader.py, synthetic.py)
| +-- features/ (engineering.py)
| +-- eda/ (eda_report.py)
| +-- models/ (train.py, tune.py, evaluate.py)
| +-- explainability/ (shap_explain.py)
| +-- fairness/ (audit.py, mitigation.py, hmda_audit.py)
| +-- utils/ (logging_utils.py)
+-- tests/ # pytest unit tests + one integration smoke test
+-- requirements.txt
+-- pyproject.toml # packaging + pytest config
+-- README.md
```

---

4. Setup Guide

4.1 Prerequisites

- Python 3.10+ (built and tested on 3.13)
- git
- ~2 GB free disk for the virtual environment + dependencies

4.2 Environment setup

```
cd Code
python -m venv .venv

# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate
```

```
pip install -r requirements.txt
pip install -e . # installs the `dac` package from src/ in editable mode
```

`requirements.txt` pins minimum versions for: `numpy`, `pandas`, `scikit-learn`, `scipy`, `xgboost`, `lightgbm`, `optuna`, `shap`, `fairlearn`, `matplotlib`, `seaborn`, `joblib`, `pyyaml`, `requests`, `pyarrow`, `pytest`, `kaggle`, `tabulate`, `jupyter`.

> **Windows note:** if `pip install` tries to compile `numpy` from source (slow, and can fail without a C/C++ toolchain), it usually means the resolver picked a version with no prebuilt wheel for your Python version. Upgrade pip first (`python -m pip install --upgrade pip`) and re-run — the pinned minimums in this repo were chosen to have prebuilt wheels on Python 3.10–3.13.

4.3 Data setup

Default (no action needed): the pipeline auto-generates and caches a schema-accurate synthetic dataset on first run.

To use real data instead:

```
# Home Credit Default Risk (Kaggle competition — requires an account that
# has accepted the competition rules, and API credentials at
# ~/.kaggle/kaggle.json, from https://www.kaggle.com/settings)
python scripts/download_home_credit.py

# HMDA (public CFPB Data Browser API, no authentication)
python scripts/download_hmda.py --year 2023 --states MI OH IN IL WI
```

No code changes are required afterward — `dac.data.loader` detects the real files under `data/raw/` and prefers them automatically. See `data/README.md` for the exact expected file layout.

4.4 Verifying the install

```
pytest
```

Expect 16 tests to pass (15 unit tests + 1 full-pipeline integration smoke test that runs the entire pipeline on a tiny synthetic sample). This is the fastest way to confirm the environment is correctly set up before a full run.

---

5. Running Guide

5.1 Full pipeline (recommended entry point)

```
python scripts/run_pipeline.py
```

Runs all ten stages described in Section 2.1 against the Home Credit and HMDA data (real if downloaded, synthetic otherwise). On the full-size synthetic dataset (25,000 / 15,000 rows) this takes roughly 10–15 minutes, most of it in the Optuna XGBoost tuning stage.

For a fast iteration/smoke-test loop (small synthetic samples, few tuning trials, ~1–2 minutes):

```
python scripts/run_pipeline.py --quick
```

Outputs:

Location	Contents
`reports/figures/home_credit/`	EDA plots, evaluation suites, SHAP plots, fairness plots
`reports/figures/hmda/`	EDA + evaluation + fairness plots for the HMDA audit
`reports/model_performance/`	`eda_*.md`, `model_comparison.csv/json/png`, `fairness/*.json`, `run_summary.json`
`models/`	`logistic_regression_tuned.joblib`, `xgboost_tuned.joblib`, `<best_model>_mitigated.joblib`

`reports/model_performance/run_summary.json` is the single consolidated record of the whole run — shapes, EDA stats, all metrics, tuning best params, and fairness results before/after mitigation.

5.2 Notebooks (interactive / exploratory)

Open with `jupyter lab` or `jupyter notebook` from the `Code/` directory:

Notebook	Covers
`01_eda_home_credit.ipynb`	Data loading + EDA suite
`02_model_training_tuning.ipynb`	Baseline training, hyperparameter tuning, evaluation
`03_explainability_shap.ipynb`	SHAP global/local explanations (loads `models/xgboost_tuned.joblib` — run notebook 02 or `run_pipeline.py` first)
`04_fairness_audit_mitigation.ipynb`	Pre-mitigation audit, reweighing, post-mitigation re-audit
`05_hmda_fairness_audit.ipynb`	HMDA EDA + fairness-only audit

Each notebook is a thin, readable wrapper around the same `dac` functions `run_pipeline.py` calls — they're for inspection and iteration, not a separate implementation.

5.3 Running individual stages from Python

Every stage is directly importable, e.g.:

```
from dac.config import CONFIG
from dac.data.loader import load_home_credit
from dac.eda.eda_report import run_eda

df, is_synthetic = load_home_credit()
stats = run_eda(
df,
target_col="TARGET",
protected_attributes=["CODE_GENDER", "AGE_GROUP"],
figures_dir=CONFIG["paths"]["figures_dir"],
report_path=CONFIG["paths"]["metrics_dir"] / "eda_home_credit.md",
dataset_name="home_credit",
)
```

5.4 Tests

```
pytest # everything
pytest tests/test_models.py -v # one module
pytest -k fairness # by keyword
pytest tests/test_pipeline.py -v -s # full-pipeline smoke test, streamed output
```

---

6. Interpreting the Outputs

- **`model_comparison.csv/json`** — ROC-AUC, PR-AUC, F1, KS statistic, and Brier score for every baseline and tuned model, side by side. The model with the highest ROC-AUC is automatically selected as "best" for the downstream SHAP/fairness/mitigation stages.
- **`eda_home_credit.md` / `eda_hmda.md`** — shape, duplicates, target balance, top-missingness columns, top correlations with target, and the target rate by protected attribute (the last table is the EDA-level early warning for the fairness audit that follows later in the pipeline).
- **`fairness_<model>_<attribute>.json`** — per-group selection rate, accuracy, TPR/FPR/FNR; demographic parity difference/ratio; equalized odds difference; and a boolean `disparate_impact_flag` (true if the demographic parity ratio falls below the configured four-fifths threshold). Compare the `_pre_mitigation` and `_post_mitigation` files for the same attribute to see the effect of reweighing.
- **`run_summary.json`** — everything above in one file, plus SHAP top features and the HMDA results, for a single-glance run record.
- **SHAP bar / beeswarm plots** — global feature importance and direction-of-effect; `EXT_SOURCE_1/2/3` (Home Credit's external credit-bureau scores) dominating is the expected, sanity-checking result.
- **Note on HMDA results:** because HMDA's origination rate is realistically high (~90%+, matching real HMDA data), the approval/denial model's raw ROC-AUC can be modest even when a real, injected fairness gap is present — a highly imbalanced favorable class compresses achievable separation. Read the fairness metrics

(selection rate by group, demographic parity ratio) as the primary signal for this pipeline, not model ROC-AUC.

---

7. Extending the Project

- **Add a model:** implement a `build_<name>_pipeline()` in `dac.models.train` returning an `sklearn.Pipeline` with steps `("preprocess", preprocessor)` and `("clf", estimator)`, add a matching `tune_<name>()` in `dac.models.tune`, and wire it into `scripts/run_pipeline.py`'s Section 4/5. Every downstream stage (SHAP, fairness audit) works unmodified as long as the pipeline follows this `preprocess` + `clf` step-naming convention.
- **Add a mitigation technique:** add a function to `dac.fairness.mitigation` (e.g. an `sklearn`-compatible adversarial debiasing or equalized-odds postprocessing wrapper) and call it from Section 9 of `run_pipeline.py` alongside `compute_reweighing_weights`.
- **Mitigate on more than one attribute:** `run_pipeline.py` currently mitigates only on `protected_attributes[0]` (`CODE_GENDER`) — this is a scope decision, not a limitation of `dac.fairness.mitigation`. To extend, compute a joint reweighing over multiple attributes' Cartesian product of groups, or run reweighing + re-audit per-attribute in a loop.
- **Swap in real data:** see Section 4.3 — no code changes needed, just run the download scripts.
- **Change any tunable parameter:** edit `config/config.yaml`; every stage reads from `dac.config.CONFIG` at call time, so no source changes are needed for path, seed, split-ratio, tuning-trial-count, or fairness-threshold changes.

---

8. Troubleshooting

Symptom	Likely cause / fix
`pip install` tries to build `numpy`/`pandas` from source and fails	No prebuilt wheel matched for your Python version. Upgrade `pip` first, then reinstall.
`KaggleApi` / `OSError: Could not find kaggle.json`	Expected if you haven't set up Kaggle credentials — the pipeline will just use synthetic data. To use real data, place `kaggle.json` in `~/.kaggle/` per Section 4.3.
`run_pipeline.py` runs but fairness plots look empty/flat	Check `reports/model_performance/fairness/*.json` for the actual per-group numbers — small synthetic samples (e.g. `--quick` mode) can have too few rows per group for a stable signal.

SHAP step is slow	It runs on `max_background=200` / `max_explain=500` rows by default (see `dac.explainability.shap_explain.explain_model`); reduce these for faster, less precise runs.
Tests fail after editing `config/config.yaml`	`tests/test_pipeline.py` monkeypatches `CONFIG` at test time — a real edit to the YAML file changes defaults for *all* runs including tests. Confirm the new values are still valid types/paths.
`ModuleNotFoundError: No module named 'dac'`	Run `pip install -e .` from `Code/` (Section 4.2) — the package must be installed in editable mode for both scripts and tests to import it.

---

9. Appendix: Configuration Reference

`config/config.yaml` keys:

Key	Meaning
`seed`	Global random seed used everywhere (data generation, splits, model init, tuning samplers).
`paths.*`	Relative paths (resolved to absolute at load time) for raw/processed data, models, reports, figures, metrics. Directories are auto-created on import.
`data.home_credit.n_synthetic_rows`	Row count for the synthetic Home Credit generator.
`data.home_credit.target_col` / `id_col` / `protected_attributes`	Column names used throughout the Home Credit pipeline.
`data.hmda.*`	Same, for HMDA.
`split.test_size` / `val_size`	Train/test split fractions (`val_size` is currently reserved; the pipeline uses a single train/test split plus CV within training).
`models.logistic_regression.max_iter`	Default LR solver iteration cap.
`models.xgboost.n_estimators` / `tree_method`	Default (untuned) XGBoost baseline params.
`tuning.n_trials`	Optuna trial count for XGBoost tuning.

`tuning.cv_folds`	Stratified K-fold count used by both LR and XGBoost tuning.
`tuning.scoring`	CV scoring metric for tuning (`roc_auc`).
`fairness.favorable_label`	Which class of the target counts as "favorable" for selection-rate purposes (`0` = repaid, for Home Credit's `TARGET`).
`fairness.disparate_impact_threshold`	The four-fifths-rule cutoff (default `0.80`).
`fairness.mitigation_method`	Currently informational; the implemented method is reweighing (`dac.fairness.mitigation`).